In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler,OneHotEncoder
from sklearn.metrics import accuracy_score,precision_score,f1_score,recall_score,con
from sklearn.tree import DecisionTreeClassifier,plot_tree
```

In [2]:
```
df=pd.read_csv('/home/backup/ml_datasets/adult_income.csv')
print(df.isnull().sum())
```

```
age                 0
workclass        2799
fnlwgt              0
education           0
education-num       0
marital-status      0
occupation       2809
relationship        0
race                0
sex                 0
capital-gain        0
capital-loss        0
hours-per-week      0
native-country    857
class               0
dtype: int64
```

In [3]:
```
print(df.head())
```

```
    age  workclass    fnlwgt     education  education-num      marital-status  \
0  25.0    Private  226802.0          11th            7.0       Never-married
1  38.0    Private   89814.0       HS-grad            9.0  Married-civ-spouse
2  28.0  Local-gov  336951.0    Assoc-acdm           12.0  Married-civ-spouse
3  44.0    Private  160323.0  Some-college           10.0  Married-civ-spouse
4  18.0        NaN  103497.0  Some-college           10.0       Never-married

          occupation relationship   race     sex  capital-gain  capital-loss  \
0  Machine-op-inspct    Own-child  Black    Male           0.0           0.0
1    Farming-fishing      Husband  White    Male           0.0           0.0
2    Protective-serv      Husband  White    Male           0.0           0.0
3  Machine-op-inspct      Husband  Black    Male        7688.0           0.0
4                NaN    Own-child  White  Female           0.0           0.0

   hours-per-week native-country  class
0            40.0  United-States  <=50K
1            50.0  United-States  <=50K
2            40.0  United-States   >50K
3            40.0  United-States   >50K
4            30.0  United-States  <=50K
```

In [4]:
```
X=df.drop('class',axis=1)
y=df['class']
```

In [5]:
```
y=y.astype(str).str.strip()
y=y.map({"<=50K":0,">50K":1})
print("\nTarget values:")
print(y.value_counts())
```

```
Target values:
0    37155
1    11687
Name: class, dtype: int64
```

In [6]:
```
# Identify numerical and categorical columns

numerical_features=X.select_dtypes(include=['int64','float64']).columns.tolist()
categorical_features=X.select_dtypes(include=['category','object']).columns.tolist()
print("\n Numerical features")
print(numerical_features)
print("\n Categorical features")
print(categorical_features)
```

```
 Numerical features
['age', 'fnlwgt', 'education-num', 'capital-gain', 'capital-loss', 'hours-per-week']

 Categorical features
['workclass', 'education', 'marital-status', 'occupation', 'relationship', 'race', 's
ex', 'native-country']
```

In [7]:
```
# numerical columns
for column in numerical_features:
    X[column]=X[column].fillna(X[column].median())
for column in categorical_features:
    X[column]=X[column].fillna(X[column].mode()[0])
```

In [8]:
```
#One hot encoding of categorical data
encoder=OneHotEncoder(handle_unknown='ignore',sparse=False)
X_categorical=encoder.fit_transform(X[categorical_features])

#Get encoded categorical features
encoded_feature_names=(encoder.get_feature_names(categorical_features))

#convert encoded data into DataFrame
X_categorical=pd.DataFrame(X_categorical,columns=encoded_feature_names,index=X.index
```

In [9]:
```
#Standard scalar for numerical data

scaler=StandardScaler()
X_numerical=scaler.fit_transform(X[numerical_features])
X_numerical=pd.DataFrame(X_numerical,columns=numerical_features,index=X.index)
print('\n Size:',X_numerical.shape)
```

```
Size: (48842, 6)
```

In [10]:
```
#Combine numerical and categorical data
X_processed=pd.concat([X_numerical,X_categorical],axis=1)
print("\n Final processed dataset shape:",X_processed.shape)
```

```
Final processed dataset shape: (48842, 105)
```

In [11]:
```
X_train,X_test,y_train,y_test=train_test_split(X_processed,y,test_size=0.2,random_st
print("Training set size:",X_train.shape[0])
print("Testing set size:",X_test.shape[0])
```

```
Training set size: 39073
Testing set size: 9769
```

In [12]:
```
#Logistic Regression
log_model=LogisticRegression(max_iter=1000,random_state=42)
log_model.fit(X_train,y_train)
log_pred=log_model.predict(X_test)
```

In [13]:
```
#Decision Tree
tree_model=DecisionTreeClassifier(criterion='entropy',max_depth=2,random_state=42)
tree_model.fit(X_train,y_train)
tree_pred=tree_model.predict(X_test)
```

In [14]:
```
#Evaluation
print("Logistic Regression\n")
print("Accuracy:",accuracy_score(y_test,log_pred))
print("Precision:",precision_score(y_test,log_pred))
print("Recall:",recall_score(y_test,log_pred))
print("F1:",f1_score(y_test,log_pred))
print("Classification Report :",classification_report(y_test,log_pred))
print("Confusion matrix:",confusion_matrix(y_test,log_pred))

print("\n")
print("Decision Tree\n")
print("Accuracy:",accuracy_score(y_test,tree_pred))
print("Precision:",precision_score(y_test,tree_pred))
print("Recall:",recall_score(y_test,tree_pred))
print("F1:",f1_score(y_test,tree_pred))
print("Classification Report :",classification_report(y_test,tree_pred))
print("Confusion matrix:",confusion_matrix(y_test,tree_pred))
```

```
Logistic Regression

Accuracy: 0.8554611526256526
Precision: 0.7340085287846482
Recall: 0.6013100436681222
F1: 0.6610657705232836
Classification Report :               precision    recall  f1-score   support

           0       0.88      0.93      0.91      7479
           1       0.73      0.60      0.66      2290

    accuracy                           0.86      9769
   macro avg       0.81      0.77      0.78      9769
weighted avg       0.85      0.86      0.85      9769

Confusion matrix: [[6980  499]
 [ 913 1377]]


Decision Tree

Accuracy: 0.8111372709591566
Precision: 0.980561555075594
Recall: 0.19825327510917032
F1: 0.32982201235016345
Classification Report :               precision    recall  f1-score   support

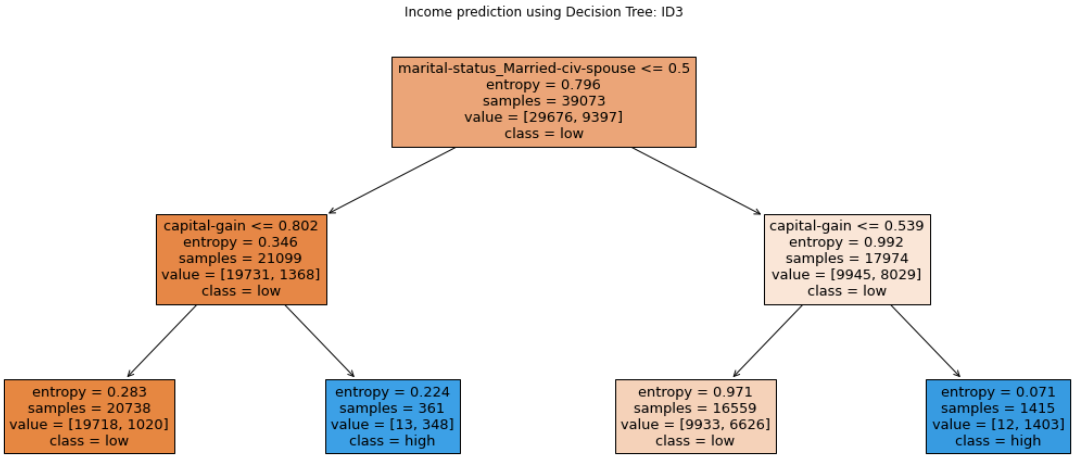
           0       0.80      1.00      0.89      7479
           1       0.98      0.20      0.33      2290

    accuracy                           0.81      9769
   macro avg       0.89      0.60      0.61      9769
weighted avg       0.84      0.81      0.76      9769
```

```
Confusion matrix: [[7470    9]
 [1836  454]]
```

In [15]:

```
plt.figure(figsize=(20,8))
plot_tree(tree_model,feature_names=X_train.columns,class_names=['low','high'],filled
plt.title("Income prediction using Decision Tree: ID3")
plt.show()
```



In [ ]: